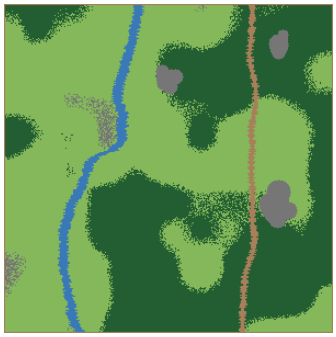
CSC2002S

PCP1 Assignment 2026

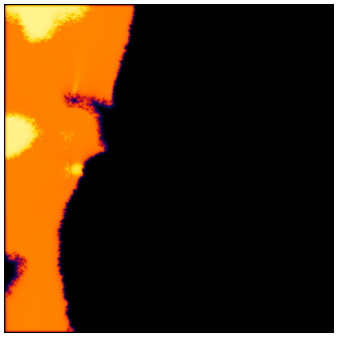
Parallel Programming with Java:

Fireline: *Parallel Heat Diffusion for a* **Wildfire** *Simulation*

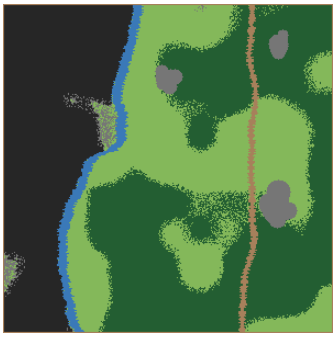
version 1, set by M. M. Kuttel with ChatGPT



Generated terrain



Peak temperature



Final burn state

1. Introduction

This first PCP assignment will give you experience in programming a parallel algorithm for the shared-memory parallel programming model using Java. You will validate your parallel solution for correctness and benchmark it against the serial algorithm supplied, to determine under which conditions parallelisation is worth the additional programming effort.

The assignment is focused on a simple, but computationally costly, simulation of a wildfire that models ignition of the fire, heat diffusion, cooling, fuel consumption and heat released by burning vegetation. This model employs a **stencil computation,** in which each point in a grid is updated from the values of nearby points. Stencil computations are a widely used class of numerical algorithm, also used in simulations of fluid dynamics and weather.

The landscape is represented as a two-dimensional grid. Each cell contains fixed terrain information and the current simulation state, including temperature, fuel and whether the cell is currently burning. At each simulation timestep, the next state of a cell is calculated from its current state and the temperatures of its four orthogonal neighbours.

north

|

west —— current cell —— east

|

south

This simulation uses parameters that give deterministic and visually interesting behaviour; this is not a predictive wildfire model.

2. The serial solution

You are given a complete Java program that implements the serial simulation. Your first task is to study the code, determine how the timestep calculation and convergence test work, and experiment with a range of inputs.

2.1 Simulation configurations

The program has two **modes**:

- diffusion: heat spreads through the landscape without combustion;
- wildfire: burnable cells may ignite, generate heat and consume fuel.

It also has two **landscape** configurations:

- **mixed**: a generated landscape containing irregular grass and forest regions, rock outcrops, a river and a firebreak;
- **grass**: a uniform grass landscape.

The **seed** provided determines which mixed landscape is generated. For the grass landscape the seed is disregarded. Unless explicit fire ignition coordinates are supplied, a default 9 x 9 ignition patch is used, depending on the selected mode.

2.2 Program arguments

The serial program takes the following command-line arguments, in order:

```
rows columns seed mode output-prefix [max-steps] [tolerance] [landscape] [ignition-top-row ignition-left-column patch-size]
```

- rows, columns: dimensions of the simulation grid;
- seed: seed used to generate a mixed landscape;
- mode: diffusion or wildfire;
- output-prefix: prefix for the generated image files;
- max-steps: optional safety limit on the number of timesteps;
- tolerance: optional convergence tolerance;
- landscape: optional mixed or grass selection (default: mixed);
- ignition-top-row, ignition-left-column, patch-size: optional coordinates and size of one square ignition patch. All three must be supplied together.

2.3 Java classes and output

The serial program has three Java source files:

- `FirelineSerial.java`: command-line processing, simulation loop and timing;
- `FireMap.java`: landscape generation, simulation state, serial timestep update and image output;
- `TerrainType.java`: fixed terrain and combustion parameters.

After the simulation completes, the program reports the input configuration, number of timesteps, whether it converged, cells burned, maximum peak temperature, final maximum temperature change and the core simulation time. The timing *excludes* image generation.

Three PNG files are written:

- <prefix>_terrain.png: fixed terrain layout;
- <prefix>_peak.png: highest temperature reached by each cell during the simulation;
- <prefix>_final.png: final terrain and burn-scar state.

2.4 Running the code

The Makefile supplied compiles with Java 11. From the top-level directory, run:

```
make run
```

You can provide other arguments, for example:

```
make run ARGS="500 500 17 wildfire output/run17 50000 0.05 mixed"
```

You should experiment with different grid dimensions, seeds, modes, landscapes, ignition locations and convergence parameters. You are responsible for selecting suitable inputs for validation and benchmarking.

3. Your parallel solution

Your task is to create a parallel version of the serial algorithm using the Java Fork/Join framework that is both **correct** and **faster**. The parallel program must be free of data races and must always produce the same results and output as the serial version. You must therefore demonstrate correctness before interpreting performance.

Your parallel version must retain the same command-line interface, console output and image output as the serial program. You may add profiling information where useful, but the original output must remain in the same format, for automarking purposes.

3.1 Restrictions

1. You must use a divide-and-conquer parallel algorithm with the Java Fork/Join framework.
2. You may use only join() as a synchronisation mechanism in your parallel algorithm. You may not use barriers, synchronized methods or blocks, locks, semaphores, atomic variables, `Future.get()` or other synchronisation mechanisms.

3. The parallel implementation must use actual fork() and join() calls in the correct order and must include a configurable sequential cutoff. The root Fork/Join task must also be awaited using `join()` rather than `invoke()` or `get()`.
4. You may not change the terrain and combustion constants or alter the model (numerical update rules) in order to obtain speedup. Your parallel version must behave the same way as the serial version in terms of the simulation.
5. Image generation and file writing may not be included in the measured core simulation time.

3.2 Specific requirements

1. Profile the serial program over a suitable range of grid dimensions and simulation configurations.
2. Implement a divide-and-conquer parallel version using the Java Fork/Join framework and a sequential cutoff.
3. Validate the parallel version, demonstrating that it implements the same simulation as the serial program.
4. Investigate and justify a suitable sequential cutoff. You may also investigate the effect of the ForkJoinPool parallelism level.
5. Benchmark the serial and parallel programs experimentally on your own computer **and** the departmental server.
6. Generate clear speedup graphs showing the conditions under which useful parallel speedup is obtained. Raw timing graphs will not be accepted.
7. Write a concise report presenting your methods, validation evidence, performance results and conclusions.

3.3 Generative AI assistance

You may use generative AI tools to assist with understanding, coding, and testing. You remain responsible for every submitted line of code, every claim and every result, and your entire report. AI-generated code must be inspected, tested and understood. You may not use AI to invent, alter or fabricate benchmark results, or to write your report.

Any fabricated results will result in zero for the entire assignment. Any student may be asked to reproduce and/or explain their results live to a tutor or lecturer. Failure to do this satisfactorily will be considered equivalent to non-completion of the assignment.

4. Report

Submit a concise report in PDF format only. The report must be no more than five pages, including graphs. Use multiple series on the same axes where this improves comparison. Your **concise** report must contain the following:

A brief **Methods** section should describe:

- your divide-and-conquer parallelisation approach in enough detail that someone else could replicate your work;
- how you avoided race conditions;
- how you determined an appropriate sequential cutoff;
- how you validated the parallel implementation, including evidence beyond one successful run;
- how you selected benchmark configurations and how you ran the benchmarks, including any shell scripts or scripting code used;
- the machine architectures tested, including processor/core information and Java version;
- the timing method, warm-up procedure, repetitions and summary statistic used;
- your use of generative AI and how its output was verified.

Results and discussion should include:

- clear evidence that the serial and parallel implementations produce equivalent results;
- **speedup graphs** for **both** your **own computer** and the **departmental server** over a suitable range of problem sizes;
- the selected simulation configurations and a justification of why they provide fair and reproducible benchmarks;
- the number of timesteps to convergence for each benchmark case, so that timing trends can be interpreted correctly;
- the effect of the sequential cutoff and, where investigated, the ForkJoinPool parallelism level;
- the range of cases for which the parallel program performs well;
- the maximum speedup obtained and how it compares with the ideal expected on each architecture;
- the reliability of the measurements, important trends, anomalies and likely explanations.

Conclusions should answer whether parallelisation is worthwhile for this problem and under what conditions.

A final **overall AI declaration** identifying the tools used, the tasks for which they were used, and how you verified or corrected their output.

5. Assignment submission requirements

Submit the assignment in two places.

1. Submit the code to the automarker as a zipped archive containing all files required to compile and run your parallel solution, the files on which it depends, your Makefile and a Git usage log saved as a text file using git log --all. (The Git log should demonstrate code/project development over at least 3 days).
2. Submit the report separately to Amathuba in PDF format.

Your parallel solution must include the following files with exactly these names:

- FirelineParallel.java
- FireMapParallel.java
- FireTask.java

The Makefile must compile with Java 11 and the following command must run the parallel solution correctly:

```
make run ARGS="500 500 17 wildfire output/run17 50000 0.05 mixed"
```

The command must compile and run the submitted parallel implementation, not the serial implementation.

Important: It is your responsibility to verify that the archive uploaded is the correct one, compiles on the departmental server and runs using the required Makefile command. A run that reaches the maximum timestep limit without convergence must be identified as such and may not be presented as an ordinary completed benchmark.

Note: Failure to follow the required filenames, command-line interface, Makefile command or submission format may prevent the automarker from testing your code.

---

*NOTE: if you do not follow these instructions exactly, or submit an incorrect assignment, or submit it to the incorrect place, your **mark will be zero**. The onus is on you to **check and double-check your final submission**.*

If you try to submit one minute after the final cut-off, this counts as no submission. Do not email about this – you need to manage your time and submit on time.

*The deadline for marking **queries** on your assignment is **one week after the return of your mark.** After this time, you may not query your mark.*